

Final project

Tomas Vemola, Miguel Angel Castro Gimenez, Urszula Matysiak

2025-12-07

1 Part 1

1.1 Implementation

SortedArray. This algorithm stores elements in a sorted array and performs a skewed binary search using parameter α to determine the split point. Instead of dividing at the midpoint, it calculates $\text{mid} = \text{left} + \lfloor (\text{right} - \text{left}) \times \alpha \rfloor$. It recursively searches left or right, tracking the best predecessor candidate until finding the largest value $\leq x$.

SearchTree. It constructs a physical tree structure using linked **Node** objects from a sorted array, rather than just traversing array elements. Then recursively builds α -balanced nodes with left/right pointers, where $\text{leftSize} = \lfloor \text{size} \times \alpha \rfloor$. Each **Node** stores its key, children references, and subtree size, enabling predecessor queries through actual tree traversal.

OtherArray. The OtherArray algorithm implements a cache-efficient predecessor search using the van Emde Boas (vEB) array layout. Initially, a SearchTree constructs an α -balanced BST providing each node's subtree size. This tree structure is then converted into a vEB array. The vEB layout organizes nodes recursively by size. First, the $\sqrt{n}$ largest nodes (by subtree size) are collected using a max-heap and stored contiguously at the array's beginning. This process recurses on remaining subtrees, repeatedly extracting the top $\sqrt{n}$ nodes by size and placing them together. This blocking strategy ensures that nodes likely to be traversed together reside in nearby memory locations, improving cache performance.

The **pred(x)** operation traverses this structure starting at index 1 (the root's value position). At each node, it compares x with the current value: if equal, return it; if smaller, follow the left index; if larger, follow the right index while tracking the current node as a candidate. The algorithm maintains min/max values for immediate boundary checks. This layout transforms tree traversal into sequential array accesses of nearby elements, significantly reducing cache misses compared to traditional pointer-based tree structures.

1.2 Tests

Test generation happens programmatically in Java code: **PBT.java** uses jqwik's **Arbitraries** framework to automatically generate random test inputs, and **ExhaustiveTestsUnder100-Elements.java** uses Java streams (**IntStream**) to programmatically create test sets.

Property-based tests (PBT) generated over 21,000 random test cases using jqwik, testing all three implementations with random sets, queries, and α values. Moreover, we used oracle testing - checking all implementations return the same results for the same inputs. The core tests validate the mathematical definition $\text{Pred}(x) = \max\{y \in S \mid y \leq x\}$. Additionally, Exhaustive tests manually validated approximately 80,000 cases for set sizes 1-99 with $\alpha \in \{0.25, 0.5, 0.75\}$, covering exact matches, queries between elements, and boundary conditions using consecutive and spaced integer sets.

All tests are executed automatically via Gradle and use random sets, queries, and α values.

1.3 Experiments

1.4 Hardware

Macbook Pro, Apple M1 Pro 10 CPU (8 performance and 2 efficiency), 32 Gb RAM, 1 TB SSD.

1.4.1 Alpha sensitivity

For the first experiment, our hypotheses are that 1) query time is lowest $\alpha = 0.3$ and increase as α approaches 0 or 1, 2) array-based structures will outperform **SearchTree** and, 3) **OtherArray** should show the best performance due to its cache-optimal blocked approach.

Experimental design for alpha sensitivity experiment:

- Performance indicator: Query time (ns)
- Factors: Algorithm implementation, balance parameter α , set of input integers of size n , set of queries to execute of size q , query distribution qd
- Levels: $n = 100,000$, $q = 500,000$, $\alpha = \{0.05, 0.1, \dots, 0.95\}$ (19 values), $imp = \{SortedArray, SearchTree, OtherArray\}$, $qd = \text{Uniform distribution}$
- Trials: 5
- Design points: possible pairs of (α, imp) , 5 trials - 57 values
- Outputs: Aggregated statistics (mean query time, standard deviation, standard error) for all design points

The input integers were generated using a seeded random number generator from numpy (`numpy.random.Generator.integers()`). The input queries were generated using the same method, bounded by the lowest and highest element in the input integers, resulting in a uniform query distribution. The results of the experiment, as shown in Figure 1, completely support hypothesis 2), as the array-based implementations outperform the tree-based **SearchTree** implementation. This is likely due to the overhead of traversing the tree nodes. As to hypothesis 1) as expected, performance in all algorithms is generally the best around $\alpha = 0.3$ due cache optimization. Lastly, 3) **OtherArray** outperforms the **SortedArray** consistently and shows great performance at (approx.) $\alpha = 0.6 - 0.8$ since its blocked approach also optimises for left-skewed binary search trees.

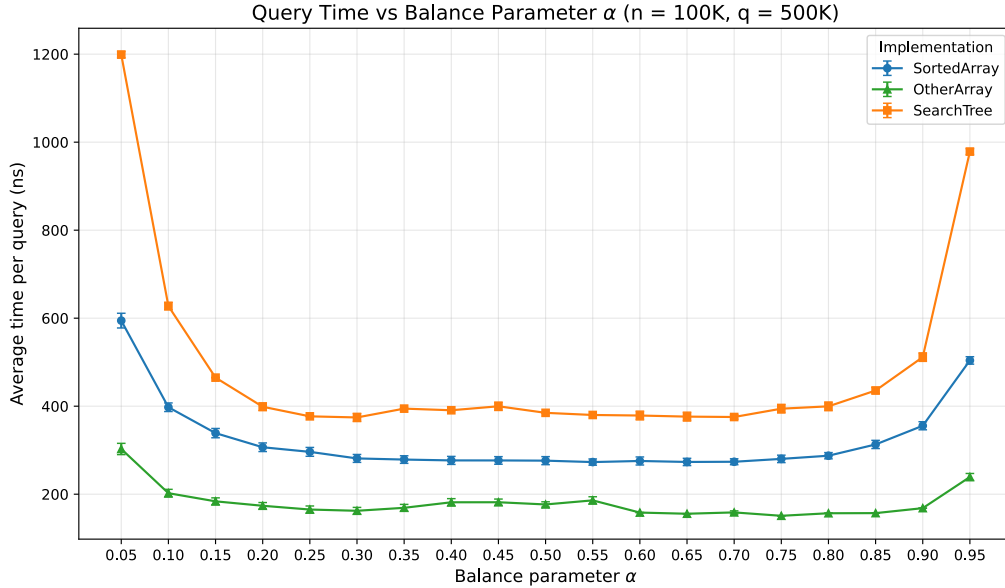


Figure 1: For border values of α , query performance degrades

1.4.2 Query locality

Our second experiment focuses on evaluating the impact of query locality (how far apart the answers are from each other). Our hypothesis is that as query locality decreases (answers are further apart), time to answer a query will increase. Our experiment design is similar to that of the previous experiment. However, we keep α constant at 0.3 (since this should be optimal for cache optimization) and instead vary the query distribution. In total, we generate data for 5 query distributions: 1) **uniform** over the range of inputs (same as previous experiment), 2)

high locality (gaps between answers are $\sim 0.1\%$ of data set size), 3) **medium locality** (gap of 1%), 4) **low locality** (gap of 10%), 5) **adversarial** (gap of 50%).

Our hypothesis is partially supported by the results (Figure 2). The query performance is best when answers are closest to each other (high locality queries). However, when queries were furthest apart (adversarial), we see better performance than with medium, uniform and low locality queries. This might be due to the fact that adversarial queries were generated by repeatedly picking values from the starting and ending 25% of the data set. Therefore, only 50% of the data set is ever queried, potentially leading to better cache optimizations. In all cases **OtherArray** performs significantly better.

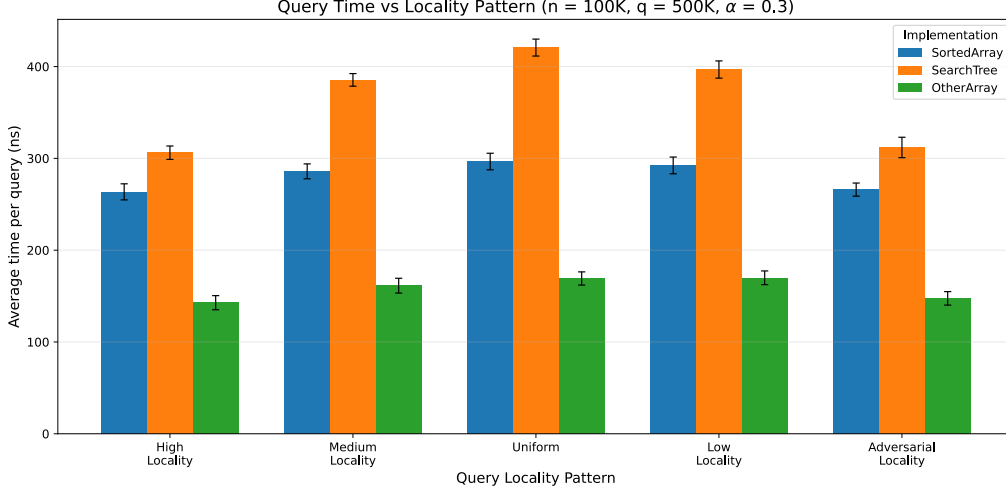


Figure 2: Query performance is best for high locality queries

2 Part 2

2.1 Implementation

RankSelectNaive. This implementation stores the original 0-1 vector directly in an `int[]` array. The `rank(i)` operation counts the number of 1s up to index i by iterating through the array, resulting in $O(n)$ query time in the worst case. The `select(r)` operation similarly scans the array until the r -th 1 is found, also taking up to $O(n)$ time. While simple to implement, this approach is inefficient for large vectors.

RankSelectLookup. In this variant, a precomputation step generates a cumulative rank array $R[]$, where $R[i]$ stores the number of 1s up to position i . The `rank(i)` query returns $R[i]$ in $O(1)$ time, while the `select(r)` query performs a binary search on $R[]$ to find the position of the r -th 1, achieving $O(\log n)$ time. This approach improves query speed but requires additional space for the rank array.

RankSelectSpaceEfficient. This space-efficient implementation stores the bit vector compactly in an `int[]` array, with 32 bits per integer. A separate array $R_s[]$ stores the rank of every “big block” of $32k$ bits. The `rank(i)` operation first determines the last big block not containing i and retrieves its precomputed rank from $R_s[b]$. It then counts and adds the rank of full 32-bit words within the next block (the one containing i), along with a masked popcount of the final partial word up until i , giving the final rank value. The `select(r)` operation proceeds in two stages. First, a binary search is performed on R_s to find the largest big block index b such that $R_s[b] < r$; this identifies the block in which the r -th 1 must lie. The remaining value $r - R_s[b]$ is then the rank that must be found inside this block. In the second stage, the algorithm scans the k packed 32-bit words of block b sequentially, subtracting the popcount of each word until

the target word is reached. Once the correct word is identified, a bit-level scan (from most significant bit to least) finds the exact position of the remaining-th 1, which is then converted to a global index.

In our implementation, we perform the popcount using `Integer.bitCount(a)`.

2.2 Tests

To test the correctness of the implementations, we created 5 test files where we tested the results manually, and when we made sure that manual tests for simple test cases with small inputs worked, we proceeded to generate larger inputs for automated tests.

Our automated test suite contains tests on random and structured vectors. Random vectors (100 cases) have lengths that are powers of two between 64 and 16,384, with bits chosen randomly. Structured vectors include all-zero, all-one, alternating, prefix-heavy, suffix-heavy, and block patterns across multiple lengths. Each vector has 20 `rank(i)` and 20 `select(r)` queries, generated automatically within valid ranges. All test cases are saved as text files and tested automatically using JUnit (in `RankSelectAutomatedTest.java` file), with the naive implementation as the correctness oracle. In total, this setup produces thousands of validated queries covering both normal and edge-case scenarios.

2.3 Experiments

2.3.1 Hardware

Macbook Air, Apple M1 8 CPU (4 performance cores and 4 efficiency), 16 Gb RAM, 256 Gb SSD.

2.3.2 Query Performance

For this experiment, our hypotheses are that 1) Naive rank and select will exhibit linear growth with input size n , 2) Lookup will achieve constant-time rank and logarithmic-time select, 3) SpaceEfficient performance will depend on parameter k , and 4) significant performance degradation will occur beyond L3 cache size. The experiment design is described in Table 1. Each trial executes 100,000 queries on a bit vector with 50% density, generated using a seeded random number generator from numpy.

The results completely support hypotheses 1) and 2) as seen in Figure 3: **Naive** times out for $n > 2^{18}$ (rank) and $n > 2^{16}$ (select), confirming linear complexity, while **Lookup** maintains near-constant rank time (22–30 ns) and logarithmic select growth. Hypothesis 3) is confirmed, with ordering $SE-k1 < SE-k4 < SE-k16$ for rank queries. Regarding hypothesis 4), a clear performance inflection occurs around $n = 2^{20}$, where **Lookup** rank time increases. An unexpected finding is that **SpaceEfficient** outperforms **Lookup** for select queries at large n . For practical applications, **Lookup** is recommended for small inputs ($n < 2^{16}$), while **SpaceEfficient** with $k = 4$ offers the best balance for large-scale workloads.

2.3.3 Query locality

For this experiment, our hypotheses are that 1) sequential access patterns will achieve optimal performance due to CPU prefetching, and 2) **SpaceEfficient** will show less sensitivity to access patterns than **Lookup** due to its smaller memory footprint. The experiment design follows Table 1 with modifications: input size is fixed at $n = 2^{24}$, query count is 500,000, **Naive** is excluded, and **SpaceEfficient** uses $k = 4$. The key factor is query pattern $qp \in \{\text{sequential, strided-64, strided-large, random, adversarial}\}$, ranging from sequential index access to adversarial patterns that alternate between distant memory regions.

Table 1: Experimental design for query performance experiment

Parameter	Value
Performance indicator	Query time (ns)
Factors	Algorithm implementation <i>imp</i> , size of input vector <i>n</i> , number of queries to execute <i>q</i> , query type <i>qp</i> , parameter <i>k</i> (for SpaceEfficient only)
Levels	<i>imp</i> = {Naive, Lookup, SpaceEfficient}, <i>n</i> = $\{2^6, 2^8, \dots, 2^{26}\}$ (11 values), <i>q</i> = 100,000, <i>qp</i> = {rank, select} <i>k</i> = {1, 4, 16}
Trials	5
Design points	All possible pairs of (<i>imp</i> , <i>n</i> , <i>qp</i> , <i>k</i>) - 110 values
Stopping rules	15 second timeout per run, larger <i>n</i> sizes for otherwise same design points are skipped
Outputs	Aggregated statistics (mean query time, standard deviation, standard error) for all design points

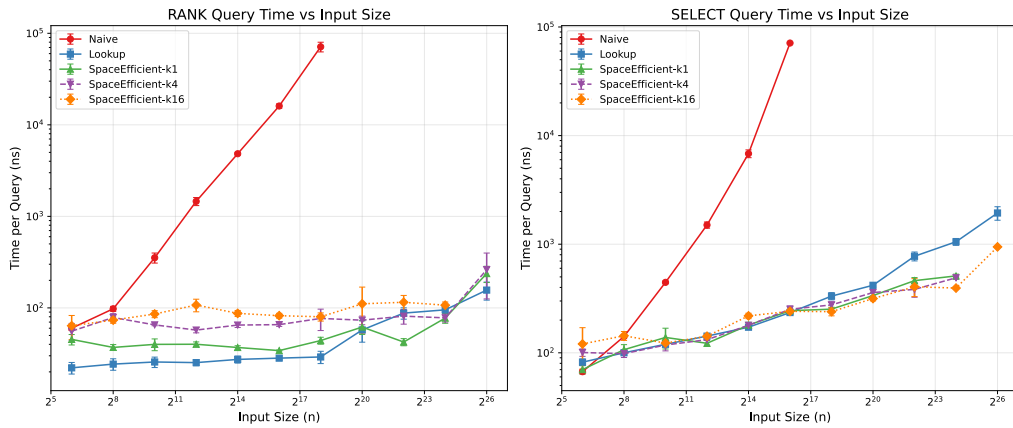


Figure 3: Naive implementation exhibits $O(n)$ performance. Lookup rank is $O(1)$ for moderate n . Lookup select follows $O(\log n)$. SpaceEfficient rank performs best with $k = 1$, while $k = 16$ handles the largest input for select queries.

The results support both hypotheses as seen in Figure 4. Although, strided (64) pattern matches the performance of the sequential pattern. This is most likely because the steps between queries are small enough to still make optimal use of caching. Poor-locality patterns cause significant degradation (see Appendix 3.1): **Lookup** select slows down $4.2\times$ on the worst performing query pattern compared to sequential access. Hypothesis 2) is strongly supported: **SpaceEfficient** select times vary only $2.7\times$ and are stable across worse performing query patterns. In comparison, **Lookup** sees $4.2\times$ slowdown for worst performing pattern, confirming that **SpaceEfficient** has reduced cache sensitivity.

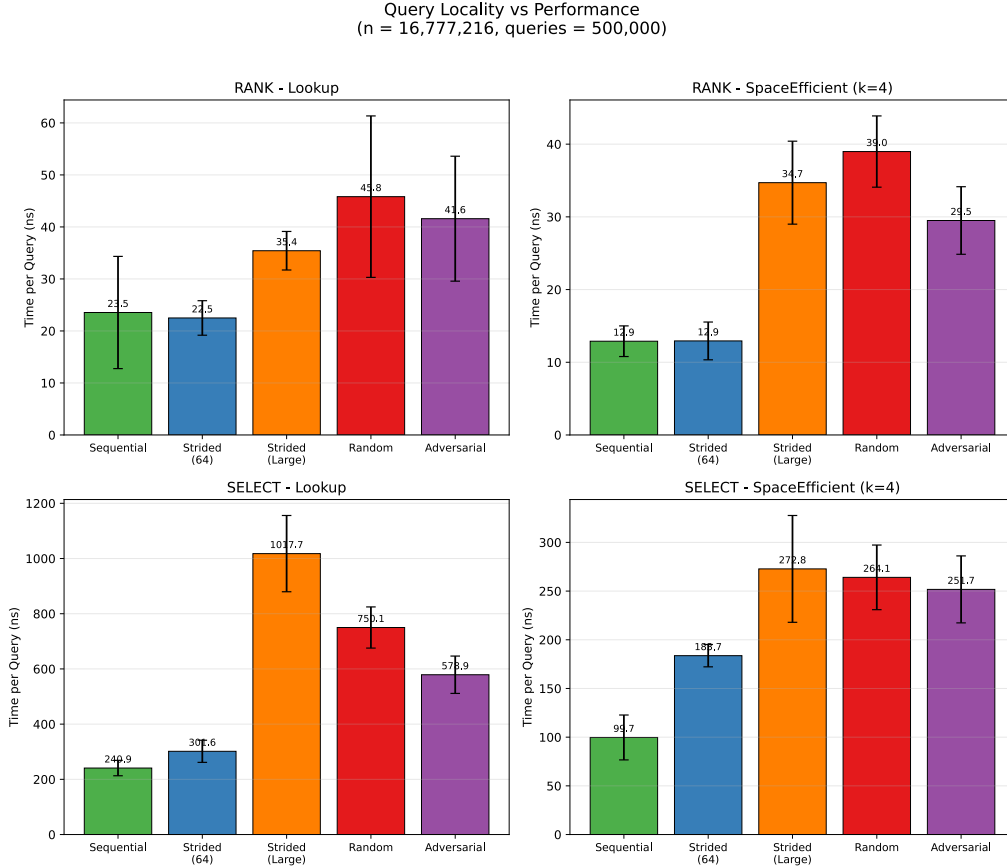


Figure 4: Query locality impact on performance. Sequential access is optimal due to prefetching. SpaceEfficient shows less cache sensitivity than Lookup due to its smaller memory footprint.

2.3.4 Precomputation

This experiment evaluates the data structure construction time for each implementation. Our hypotheses are that 1) all implementations will exhibit linear growth with input size n due to the need to process all bits, 2) Naive will be fastest as it only copies the input array, 3) Lookup will be slightly slower due to prefix sum computation, and 4) SpaceEfficient will be slowest due to bit packing overhead, with parameter k having minimal effect on construction time.

The experiment measures construction time using Java’s `System.nanoTime()` across 11 input sizes $n = \{2^6, 2^8, \dots, 2^{26}\}$ for three implementations (Naive, Lookup, SpaceEfficient) and three k values $\{1, 4, 16\}$ for SpaceEfficient. Each configuration is run for 10 trials with garbage collection forced between trials to avoid interference. Timeout is set to 60 seconds. Bit vectors are generated with 50% density using a seeded random number generator.

The results in Figure 5 partially support the hypotheses. Hypothesis 1) is confirmed for Lookup and SpaceEfficient, which show clear linear growth with n . However, Naive construc-

tion time remains nearly constant across most input sizes, contradicting hypothesis 2)—this is because Naive merely stores a reference to the input array rather than copying it. Hypothesis 3) is confirmed: Lookup grows linearly from ≈ 0.5 ms at $n = 2^6$ to ≈ 94 ms at $n = 2^{26}$. Hypothesis 4) also holds, as SpaceEfficient is slowest than Lookup, reaching ≈ 450 – 550 ms at $n = 2^{26}$ due to the overhead of bit packing operations. As expected, parameter k has negligible effect on construction time, since the sparse prefix sum array Rs is small relative to the bit vector. This trade-off is important for practical applications: SpaceEfficient offers superior query performance at large scales (Section 2.3.2) but incurs 5 – $6\times$ higher construction cost compared to Lookup.

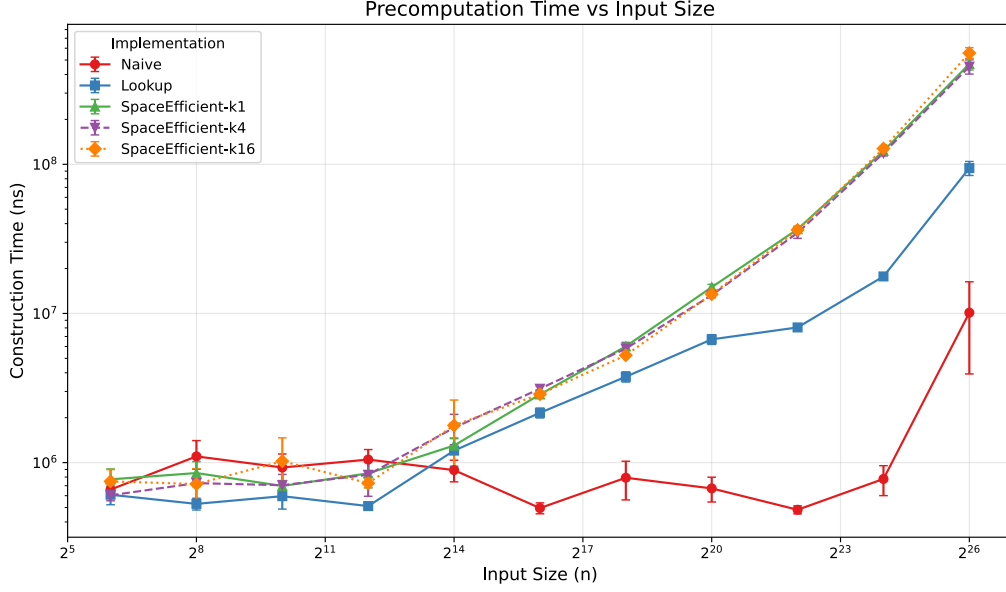


Figure 5: Construction time vs input size. Naive remains nearly constant as it stores only a reference. Lookup shows linear growth. SpaceEfficient is slowest due to bit packing overhead, with k having minimal effect.

3 Appendix

3.1 Part 2 - query locality slowdown relative

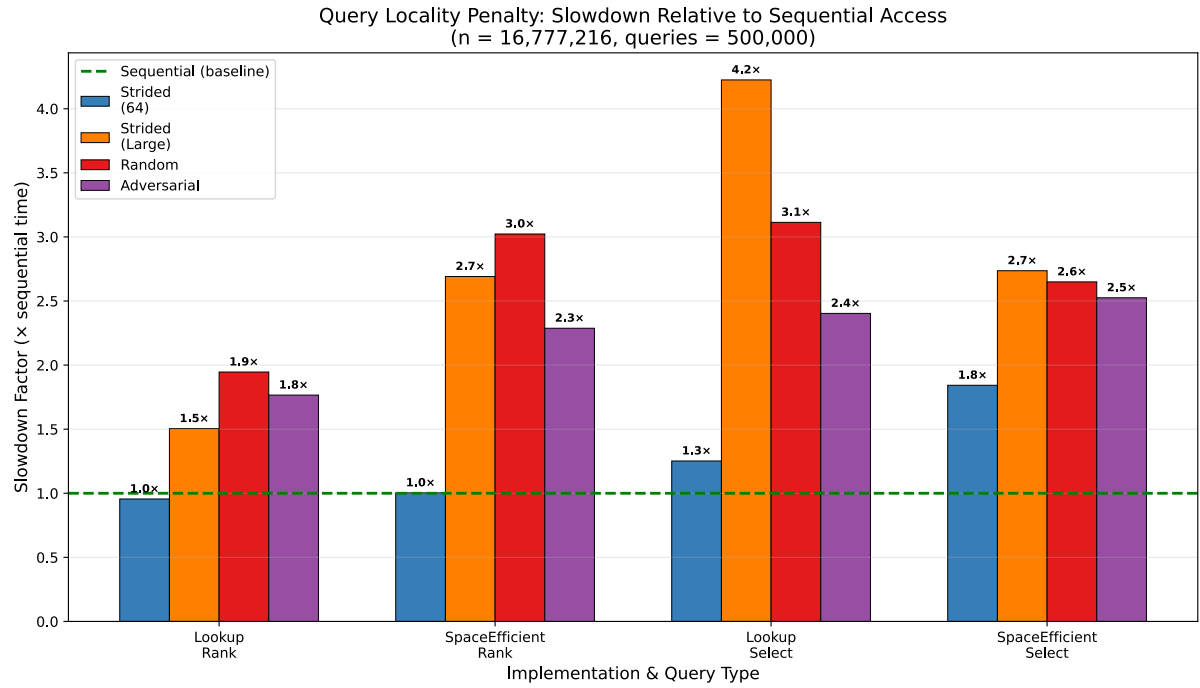


Figure 6: Query locality impact on performance, relative comparison. Non-sequential query patterns result in increase of query time.